

CODERAXO

Web Development Internship Program

PROGRESS REPORT

Weeks 4 – 6

Intern Name: Aqsa Aslam

Offer Reference: CAX-OL-2026-310

Reporting Period: July 6 – July 26, 2026

Track: Advanced JavaScript → Node.js/Express → React (MERN Final Project Planning)

Submitted as part of the CoderAxo Web Development Internship weekly reporting cycle.

Executive Summary

Weeks 4 through 6 marked the shift from static front-end work into full-stack development. Week 4 pushed vanilla JavaScript further with Task 2, a fully interactive To-Do application featuring persistence and theming. Week 5 crossed into server-side development with Node.js and Express. Week 6 introduced React and closed with the release of the Final Project brief — a full MERN-stack Project & Team Collaboration System due in a 10-day window.

By the end of Week 6, the front-end, back-end, and React fundamentals needed for the Final Project were all in place, and a scheduled, module-by-module build plan had been drafted ahead of active development beginning in Week 7.

Objectives Covered

- Build a full CRUD-style application in vanilla JavaScript with persistent client-side storage
- Understand the server-side request-response cycle and Node's core modules
- Build REST routes and middleware in Express
- Learn React's component model — components, props, state, and side effects
- Translate the Final Project brief into a realistic, scheduled build plan before coding begins

Weekly Breakdown

Week	Dates	Focus Area	Key Deliverable
Week 4	Jul 6 – Jul 12	Task 2 – To-Do App & Advanced JS	Full CRUD To-Do app with localStorage & theme toggle
Week 5	Jul 13 – Jul 19	Node.js, npm & Express Basics	HTTP server from scratch + Express REST routes
Week 6	Jul 20 – Jul 26	React Fundamentals & Final Project Briefing	Final Project brief received; MERN build plan drafted

Week-by-Week Detail

Week 4 — Task 2: To-Do Application & Advanced JS

Task 2 required task creation/deletion, a completed-state toggle, persistent storage via `localStorage`, a dark/light theme toggle, and a responsive UI. The main technical challenge was keeping the DOM, the in-memory task array, and `localStorage` in sync; this was resolved by rewriting the render logic so any state change triggers one function that fully re-renders the list from the array and then saves it, establishing a 'single source of truth' pattern that carried forward into later, backend-connected work.

Week 5 — Introduction to Node.js, npm & Express Basics

This week shifted focus from the browser to the server: how Node.js differs from browser JavaScript, npm and `package.json`, and core built-in modules (`fs`, `path`, `http`). A basic HTTP server was built from scratch before introducing Express, which made it clear what the framework does under the hood. Express routes, middleware, and a small in-memory 'notes' API were built and tested with Postman, alongside a review of REST conventions.

Week 6 — React Fundamentals & Final Project Briefing

React sessions covered components, JSX, props, `useState`, `useEffect`, lifting state up, controlled forms, and React Router. Midway through the week, CoderAxo released the Final Project brief: a Project & Team Collaboration System on the mandatory MERN stack, with a strict 10-day build window ending August 5. The remainder of the week was spent reading the requirements closely and planning — sketching the MongoDB schema (Users, Teams, Projects, Tasks) and deciding to build the project as vertical slices, one complete module (backend + frontend together) at a time.

Tools & Technologies Used

- Vanilla JavaScript (ES6), CSS custom properties, browser `localStorage`
- Node.js, npm, Express.js, Postman, VS Code REST Client
- React 18, Vite, React Router
- MongoDB Compass and Draw.io for schema and ER-diagram planning

Key Challenges & Resolutions

- Task list occasionally out of sync with localStorage — fixed by centralizing all updates through one re-render-from-state function instead of patching the DOM directly.
- Deleting the wrong to-do item after reordering — caused by deleting by array index; fixed by switching to a generated unique ID per task.
- Express route not receiving a parsed JSON body — traced to express.json() middleware being registered after the routes instead of before them.
- Recurring useEffect bugs from missing dependencies — resolved by studying React's 'rules of hooks' rather than relying on intuition.

Reflection

This stretch reframed front-end and back-end as two sides of one request-response conversation rather than separate subjects, which made the move into React and full-stack integration feel far less intimidating. Receiving the Final Project brief mid-way through learning React added pressure but also gave every new hook and pattern an immediate, concrete purpose, which noticeably improved retention. Closing the period with a deliberate, schema-first, vertical-slice plan — rather than diving straight into code — reflects a planning habit that has held steady since Week 3's portfolio project.

Goals Going Forward

- Begin active development on the Final Project's Authentication module, following the vertical-slice plan
- Start Task 3 (a deployed React app)
- Keep the 10-day Final Project deadline (August 5) firmly in view